

Exercice : Composition de deux poses

Consigne

Écrire la composition de deux poses, puis vérifier sur un cas que composer puis appliquer donne le même résultat qu'appliquer l'une après l'autre. Afficher les deux points obtenus et leur écart.

Rappel

Deux poses se composent : la main est à telle place dans le repère du coude, lui-même à telle place dans celui de l'épaule. L'ordre est toujours le même : le parent d'abord, l'enfant ensuite.

Pour deux poses $P_1 = (t_1, q_1)$ et $P_2 = (t_2, q_2)$, la composition $P = P_1 \circ P_2$ (appliquer P_2 dans le repère de P_1) est :

$$q = q_1 \cdot q_2$$
$$t = t_1 + R_{q_1} \cdot t_2$$

Fonction de composition

```
// Produit de deux quaternions unitaires.
Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}

// Composition de deux poses : parent d'abord, enfant ensuite.
Pose Composer(const Pose& parent, const Pose& enfant)
{
    Quat q = Produit(parent.orientation, enfant.orientation);
    Vec3 t = {
        parent.position.x + Rotation(parent.orientation, enfant.position).x,
        parent.position.y + Rotation(parent.orientation, enfant.position).y,
        parent.position.z + Rotation(parent.orientation, enfant.position).z
    };
    return { t, q };
}
```

Programme complet

```
#include <array>
#include <cstdio>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };

Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
```

```

        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
    return {
        v.x + 2.0f * t.x,
        v.y + 2.0f * t.y,
        v.z + 2.0f * t.z
    };
}

Vec3 Appliquer(const Pose& pose, const Vec3& p)
{
    Vec3 tourne = Rotation(pose.orientation, p);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

Quat Produit(const Quat& a, const Quat& b)
{
    return {
        a.w * b.w - a.x * b.x - a.y * b.y - a.z * b.z,
        a.w * b.x + a.x * b.w + a.y * b.z - a.z * b.y,
        a.w * b.y - a.x * b.z + a.y * b.w + a.z * b.x,
        a.w * b.z + a.x * b.y - a.y * b.x + a.z * b.w
    };
}

Pose Composer(const Pose& parent, const Pose& enfant)
{
    Quat q = Produit(parent.orientation, enfant.orientation);
    Vec3 tp = Rotation(parent.orientation, enfant.position);
    Vec3 t = {
        parent.position.x + tp.x,
        parent.position.y + tp.y,
        parent.position.z + tp.z
    };
    return { t, q };
}

int main()
{
    Pose p1, p2;
    Vec3 point;

    std::scanf("%f %f %f", &p1.position.x,
        &p1.position.y,
        &p1.position.z);
    std::scanf("%f %f %f %f", &p1.orientation.w,
        &p1.orientation.x,
        &p1.orientation.y,
        &p1.orientation.z);
}

```

```

std::scanf("%f %f %f", &p2.position.x,
               &p2.position.y,
               &p2.position.z);
std::scanf("%f %f %f %f", &p2.orientation.w,
               &p2.orientation.x,
               &p2.orientation.y,
               &p2.orientation.z);

std::scanf("%f %f %f", &point.x, &point.y, &point.z);

// Methode 1 : composer puis appliquer.
Pose composee = Composer(p1, p2);
Vec3 r1 = Appliquer(composee, point);

// Methode 2 : appliquer l'une apres l'autre.
Vec3 intermediaire = Appliquer(p2, point);
Vec3 r2 = Appliquer(p1, intermediaire);

Vec3 ecart = {
    r1.x - r2.x,
    r1.y - r2.y,
    r1.z - r2.z
};

std::printf("Composer puis appliquer    : %.4f %.4f %.4f\n",
            r1.x, r1.y, r1.z);
std::printf("Appliquer l'une apres l'autre : %.4f %.4f %.4f\n",
            r2.x, r2.y, r2.z);
std::printf("Ecart                      : %.6f %.6f %.6f\n",
            ecart.x, ecart.y, ecart.z);

return 0;
}

```

Exemple d'exécution

Pour l'entrée :

```

1.0 0.0 0.0
0.7071 0.0 0.7071 0.0
0.0 1.0 0.0
1.0 0.0 0.0 0.0
1.0 1.0 1.0

```

La pose P_1 est une translation de (1,0,0) et une rotation de 90 autour de y . La pose P_2 est une translation de (0,1,0) sans rotation.

La sortie est :

```

Composer puis appliquer    : 1.0000 2.0000 0.0000
Appliquer l'une apres l'autre : 1.0000 2.0000 0.0000
Ecart                      : 0.000000 0.000000 0.000000

```

L'écart est nul aux arrondis près, ce qui confirme que la composition est correcte.

Pourquoi la composition fonctionne

Appliquer P_2 puis P_1 à un point p donne :

$$P_1(P_2(p)) = q_1 \cdot (q_2 \cdot p + t_2) + t_1$$

En développant :

$$= q_1 \cdot q_2 \cdot p + q_1 \cdot t_2 + t_1$$

Or la pose composée $P = P_1 \circ P_2$ a pour quaternion $q = q_1 \cdot q_2$ et pour position $t = t_1 + R_{q_1} \cdot t_2$. En appliquant P à p :

$$P(p) = q \cdot p + t = q_1 \cdot q_2 \cdot p + t_1 + q_1 \cdot t_2$$

C'est exactement le même résultat. La composition est donc cohérente avec l'application successive.

À retenir

L'ordre de composition est toujours le même : le parent d'abord, l'enfant ensuite. Faites tourner l'épaule, et la main suit sans qu'on ait rien à lui dire. La composition de deux poses n'est pas commutative : $P_1 \circ P_2 \neq P_2 \circ P_1$ en général.